

Parallel Computing Basics

Parallel Computer Architecture

Parallel processor architectures

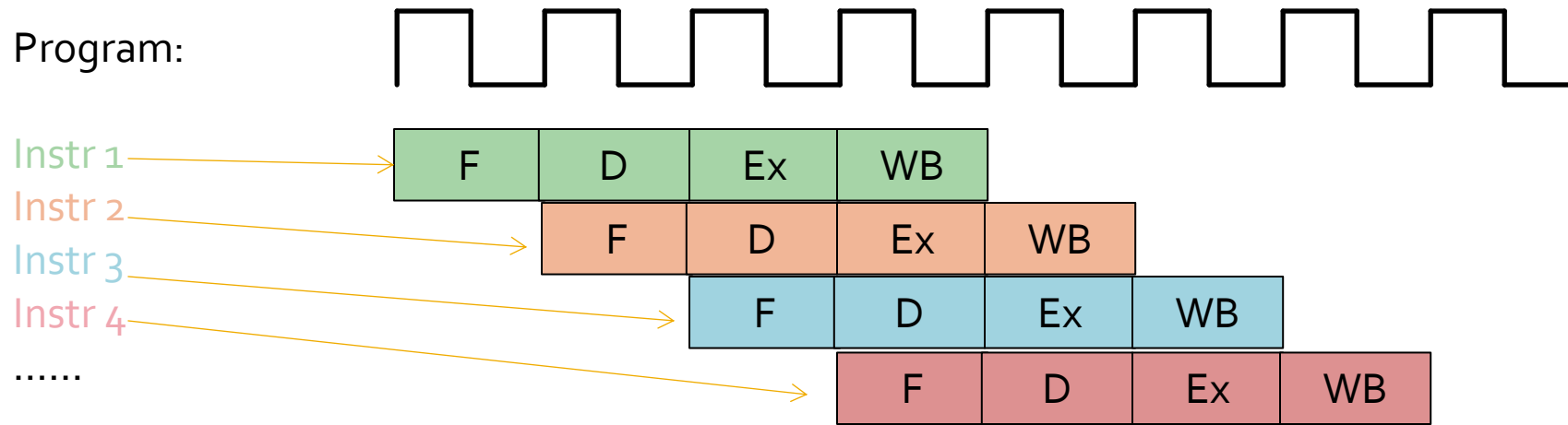
Processor architecture evolution

- Until 2006, performance gain through:
 - Frequency scaling (Moore's Law)
 - Instruction level parallelism (ILP)
- Since: 2006
 - Multicore CPUs
 - SIMD execution in GPUs

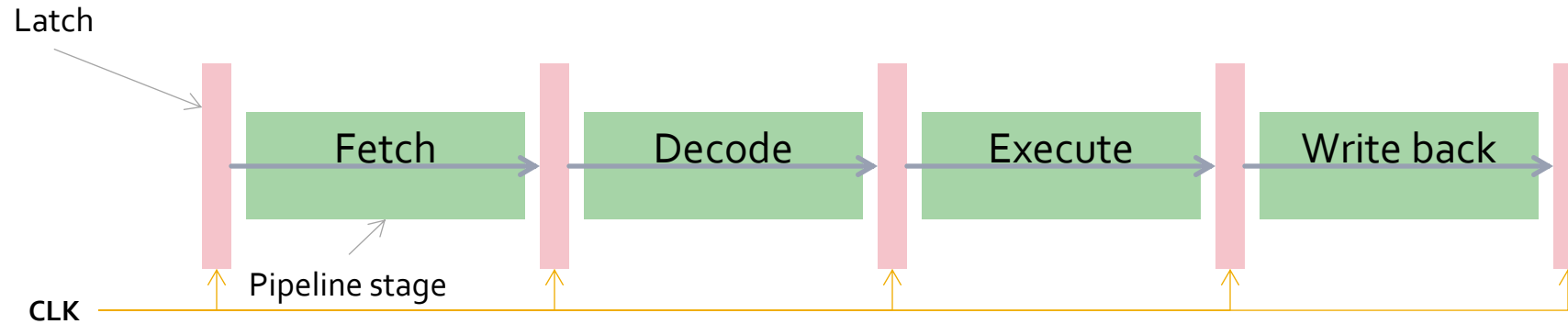
Processor: pre multicore

- Big L1 cache
 - Memory pre-fetch
 - Branch predictor
 - Out-of-order execution
-
- Sophisticated processor logic – requires more transistors

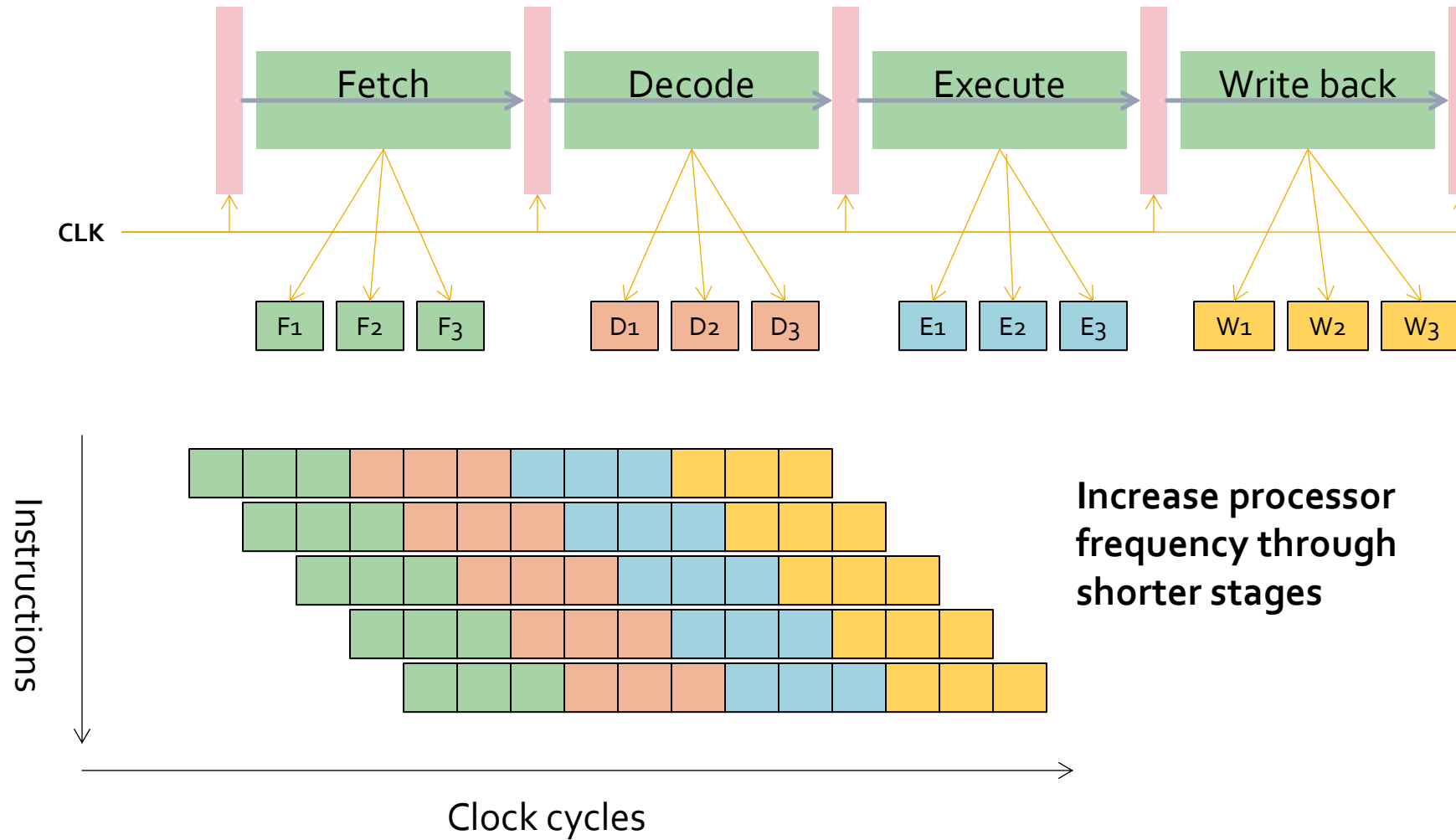
Pipeline (ILP)



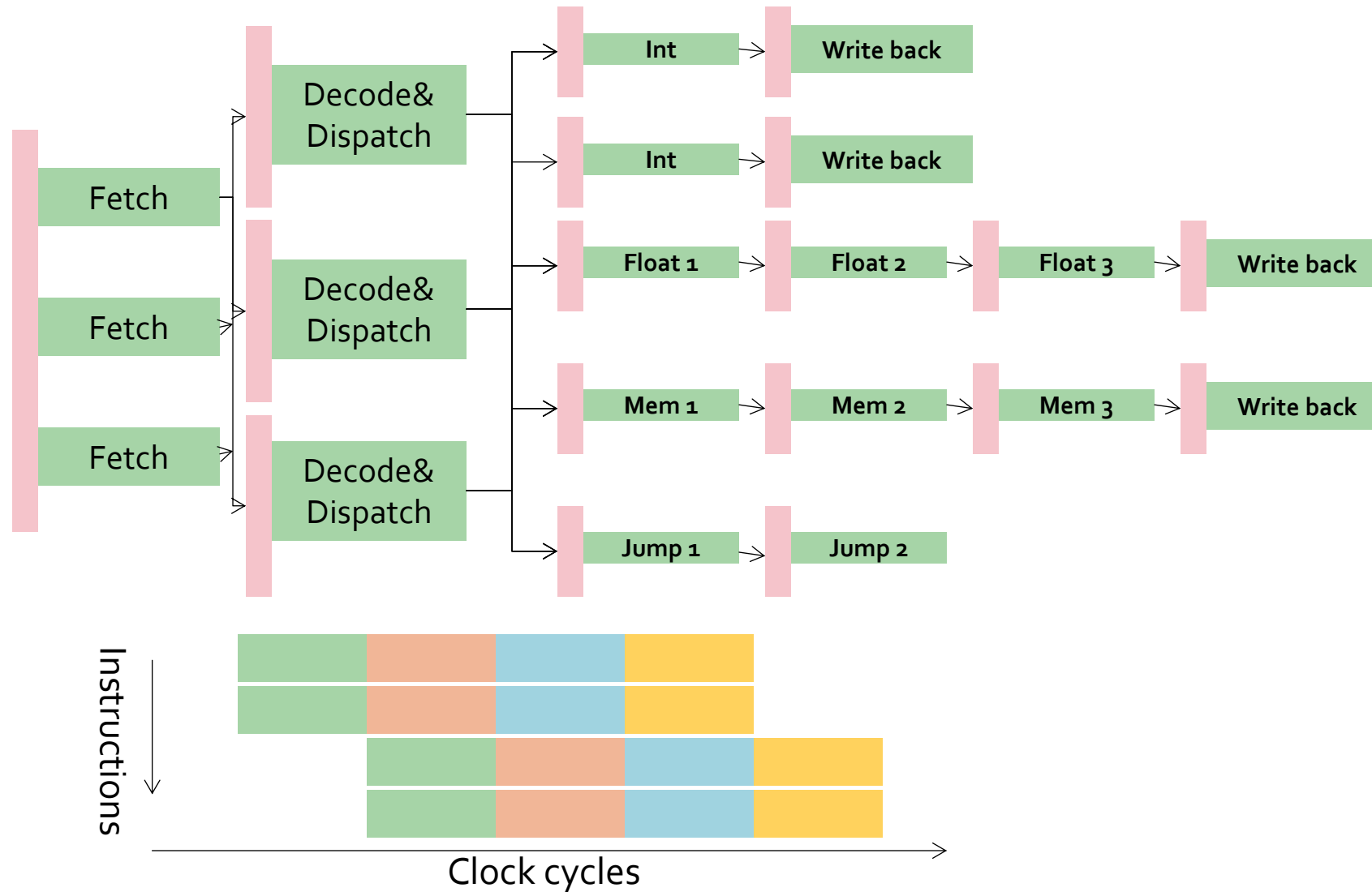
CPI=1? Hazard situations!



Superpipeline



Superscalar (ILP)



Parallel processor trends

- Multicore processors:
 - identical processing cores, integrated multicore GPU in the same chip (ex: Intel Skylake)
 - non-identical processing cores: (ex: Intel Alder Lake, Raptor Lake)
- Multicore microcontrollers
- GPUs: multiple processing blocks (ex: NVIDIA Maxwell GTX 980 GPU)
- Mobile parallel processing: multiple non-identical processing cores + multicore GPU (ex: Exynos 8890 -4 Cortex A53 cores, 4 Exynos M1 cores, 12 core Mali GPU)
- Parallelism and HW specialization: FPGAs, NPUs, GPNPUs
- Supercomputers: clusters of multi-core CPUs + GPUs

Multicores

- Use more transistors to add more cores to processors
- Use less of that sophisticated processor logic => simpler cores
- Even if cores are slower, there is a potential speedup (ex: 2 x 25% slower cores -> speedup $2 * 0.75 = 1.5$)
- Programs
 - Issue: if the program doesn't express parallelism, there is NO speedup
 - Solution: use threads!

SIMD execution

- Add ALUs to increase compute capability
- SIMD computing: single instruction, multiple data (all ALUs execute the same instructions in parallel)
- Programs
 - Special instructions for SIMD processing (ex: SSE, AVX instructions)
 - Special language constructions (ex: parallel for) transformed by compiler in low level SIMD instructions
 - Issues: efficient only if the same instruction sequence applies to all elements operated upon simultaneously (**instruction stream coherence**)

Hardware-supported multi-threading

- Core manages execution contexts for multiple threads
 - Runs instructions from runnable threads
 - Makes decision about which thread to run each clock
- Interleaved multi-threading
- Fine-grained multi-threading
- Simultaneous multi-threading
- Issues:
 - additional storage for thread context
 - Increased run time for threads

Interleaved Multi-Threading

- **How it works:**

- The processor switches between threads only when one thread stalls (e.g., due to a cache miss or memory access).
- Each thread runs until it encounters a long-latency event, at which point another thread takes over execution.

- **Characteristics:**

- Reduces idle time by utilizing processor cycles that would otherwise be wasted.
- Only one thread executes at a time per core.
- Context switching introduces some overhead.

- **Example:** Early IBM processors and some embedded processors.

Fine-Grained Multi-Threading

- **How it works:**

- The processor switches between threads every cycle, even if the current thread has not stalled.
- Ensures that different threads get execution time in a round-robin fashion.

- **Characteristics:**

- Reduces the impact of pipeline stalls by always having another thread ready to execute.
- Avoids long stalls but may introduce overhead due to frequent context switching.
- Less efficient than SMT when a single thread can fully utilize resources.

- **Example:** Used in Sun/Oracle's Niagara processors for high-throughput workloads.

Simultaneous Multi-Threading (SMT)

- **How it works:**

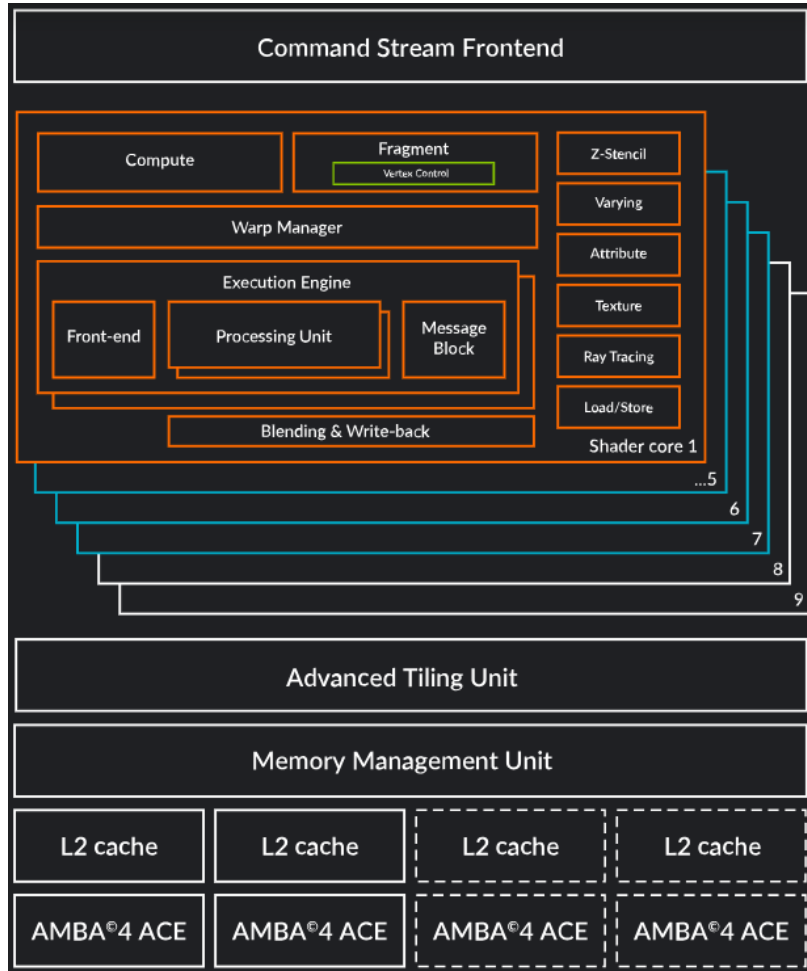
- Multiple threads execute simultaneously on the same core, sharing execution units (ALUs, FPUs, etc.).
- The processor issues instructions from multiple threads in the same cycle, maximizing resource utilization.

- **Characteristics:**

- Increases throughput by allowing multiple threads to execute in parallel.
- Can improve performance if workloads have different resource needs.
- Requires sophisticated hardware to handle dependencies and scheduling.

- **Example:** Intel's Hyper-Threading and AMD's Simultaneous Multi-Threading in modern CPUs.

Graphics Processing Units (GPUs)



ARM MALI-G720

- Optimize graphics pipeline computations
- Independent computations are executed in parallel
- SIMD, SIMT processing
- Optimize performance and efficiency across all mobile gaming applications

Features of GPUs for parallel computing

- **Massive parallelism**
 - Thousands of cores designed for high-throughput computations
 - Executes thousands of threads simultaneously
- **SIMD & SIMT execution model**
 - Single Instruction, Multiple Data (SIMD): Same operation applied to multiple data points
 - Single Instruction, Multiple Threads (SIMT): Threads grouped into warps, executing in lockstep
- **High memory bandwidth:** Optimized for fast data transfers between memory and processing cores
- **Hierarchical memory architecture**
 - Global memory (large, but slower)
 - Shared memory (fast, accessible within thread blocks)
 - Registers & caches for low-latency access
- **Specialized for compute-intensive workloads**
 - Ideal for AI, deep learning, scientific simulations, and graphics processing
 - Supports CUDA (NVIDIA) and OpenCL for parallel programming

Characteristics of SIMT

- **Warp-based execution**

- Threads are grouped into warps (32 threads per warp in NVIDIA GPUs).
- All threads in a warp execute the same instruction at the same time.

- **Independent thread processing**

- (unlike SIMD) Allows each thread to access different memory locations and use different registers.
- This makes it more flexible than SIMD while still benefiting from parallel execution.

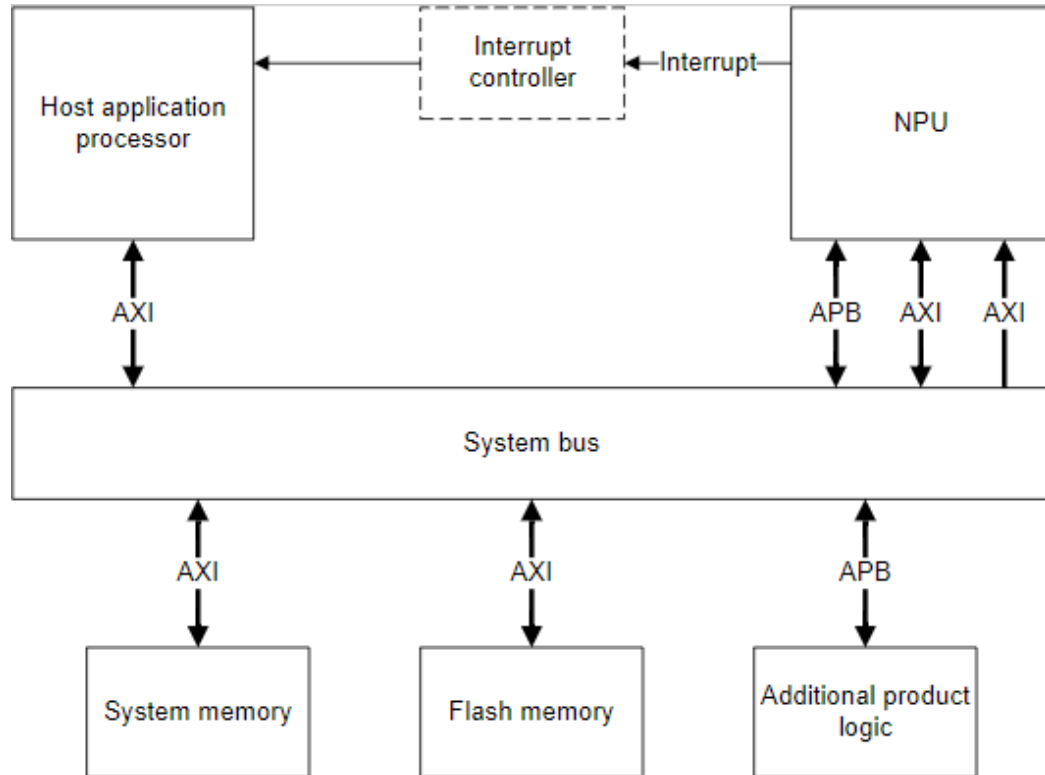
- **Efficient parallelism**

- Threads execute in lockstep, meaning they perform the same operation but on different data points.
- This is ideal for vectorized operations, such as graphics rendering, deep learning, and scientific computing.

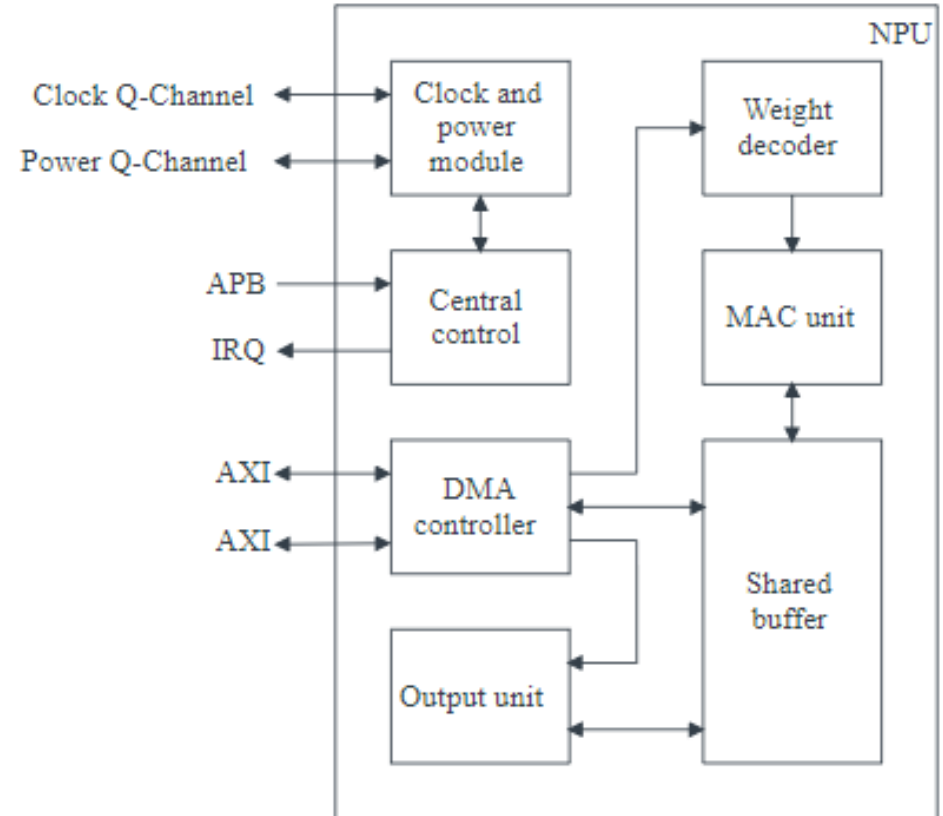
- **Warp divergence**

- If threads within a warp follow different execution paths (e.g., an if-else branch), the warp diverges.
- Divergence causes serialization, where different paths execute sequentially instead of in parallel.
- Optimizing GPU code to avoid divergence improves efficiency

NPU core from ARM (Ethos-U55)



Typical system configuration block diagram



Main components of the NPU

NPU core from ARM (Ethos-U55)

- NPU = Application-Specific Integrated Circuit (ASIC) that improves the inference performance of neural networks
- 8-bit and 16-bit integer quantized Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN), 8-bit weights
- Neural network must be compiled offline using the open-source compiler to produce a command stream
- When the NPU performs inferences, the DMA controller reads the neural network description (command stream, network weights, bias information, scale information)

Features of NPUs for parallel computing

■ **Optimized for AI Workloads**

- Designed specifically for neural network inference and training.
- Supports operations like matrix multiplications, convolutions, and activation functions used in deep learning.
- Delivers higher efficiency and lower power consumption compared to general-purpose GPUs and CPUs.

■ **High Parallelism & Throughput**

- Execute thousands to millions of operations in parallel.
- Uses massively parallel architectures similar to GPUs but with AI-specific optimizations.
- Supports batch processing to handle multiple inputs simultaneously, improving efficiency.

■ **Specialized Compute Units**

- Tensor Processing Units (TPUs) or AI Cores accelerate tensor-based operations.
- MAC (Multiply-Accumulate) Units optimize deep learning computations, especially in CNNs and transformers.
- Some NPUs integrate systolic arrays, which perform efficient matrix multiplications in hardware.

■ **Efficient Memory Architecture**

- On-chip SRAM for high-speed access to model weights and activations.
- Low-latency data movement to prevent bottlenecks in deep learning computations.
- Memory compression techniques (e.g., weight pruning and quantization) reduce bandwidth usage.

■ **Low Power Consumption**

- Optimized for edge AI devices like smartphones, IoT, and autonomous systems.
- Consumes far less power than GPUs while delivering high AI performance.

■ **AI Framework Support & Customization**

■ **Scalability & Cloud Deployment**

- NPUs are used in data centres for large-scale AI workloads (e.g., Google TPUs in Google Cloud).
- Scalable across multiple chips for distributed AI training.
- Efficient for real-time applications like autonomous driving, NLP, and video analytics.

NPU vs GPU vs multicore CPUs

Feature	NPU (Neural Processing Unit)	GPU (Graphics Processing Unit)	Multi-Core CPU
Purpose	Optimized for AI/ML workloads (inference & training)	Designed for parallel computing & graphics processing	General-purpose computing with multi-threading
Architecture	Specialized matrix/tensor processing units	Many cores optimized for parallelism	Fewer but powerful cores with complex logic
Performance	High efficiency for AI tasks (low power, high throughput)	High parallelism, good for AI, gaming, and simulations	Good for sequential and parallel tasks but less optimized for AI
Power Efficiency	Very high (optimized for energy-efficient AI processing)	Moderate (power-hungry but efficient in parallel tasks)	Lower efficiency for AI, high for general workloads

FPGAs for Parallel Computing

- **Field-Programmable Gate Arrays (FPGAs)** = integrated circuits that can be configured by the user, can be reprogrammed to perform different tasks, making them highly versatile.
- **Features**
 - Parallel Processing: FPGAs can execute multiple operations simultaneously, making them ideal for parallel computing.
 - Customizability: Users can design custom hardware architectures tailored to specific applications.
 - Low Latency: FPGAs offer low-latency processing, which is ideal for real-time applications.
- **Applications**
 - High-Performance Computing (HPC): Used in supercomputers for tasks requiring massive parallelism.
 - Data Centres: Enhance the performance of data-intensive applications.
 - Embedded Systems: Commonly used in automotive, aerospace, and industrial applications for real-time processing.
- **Benefits**
 - Flexibility: can be reprogrammed
 - Efficiency: Optimized for specific tasks, leading to higher efficiency compared to general-purpose processors.
 - Scalability: Can be scaled to handle large and complex computations.
 - Energy Efficiency: Often more energy-efficient than traditional processors for certain tasks.
- **Example Use Cases**
 - Cryptography: Accelerating encryption and decryption processes.
 - Machine Learning: Implementing custom neural network architectures.
 - Databases: Accelerating query processing.
 - Signal Processing: Real-time processing of audio, video, and communication signals.

Coarse-Grained Reconfigurable Architectures (CGRAs)

- 2D array of **word-level processing elements (PEs)** — operating on 8, 16, or 32-bit data — connected through a reconfigurable interconnect network
- A **configuration bitstream** programs the dataflow — which PE does what operation and how data flows between them
- CGRAs are gaining significant attention for AI/ML acceleration
- Situated between FPGAs and ASICs in terms of flexibility and efficiency

Dimension	ASIC	CGRA	FPGA
Performance	Best possible — every transistor optimized for the target task. No overhead.	Excellent for regular dataflow (ML, DSP). Loses edge on irregular tasks.	Good for parallel tasks but overhead from LUT-based logic and routing limits peak performance.
Flexibility	Zero — fixed at fabrication.	Medium — can remap PE operations and dataflow in nanoseconds.	Maximum — can implement any digital circuit.
Energy Efficiency	Best — 1–10 TOPS/W for edge ASICs, 100+ TOPS/W for advanced nodes.	Very good — 5–10× better than FPGA.	Weakest — 1–5 TOPS/W.
Cost (NRE)	Extremely high	Moderate.	Negligible.
Design Complexity	Hardest — requires full custom RTL, physical design, DRC/LVS, tape-out. Years of expert work.	Medium — dataflow/spatial compilers handle mapping. Less mature tooling than FPGA.	Medium — HLS tools (Vitis, Intel HLS) allow C/C++ design. Large ecosystem and IP libraries available.
Reconfigurability	None after fabrication.	Cycle-level reconfiguration — switch between tasks in nanoseconds.	Partial dynamic reconfiguration possible but slow (ms range). Full reconfiguration takes seconds.
Prototyping Speed	Years from concept to working silicon.	Months for new mapping on existing fabric.	Days to weeks from algorithm to working hardware.
Best Use Case	High-volume, fixed-function products: smartphones, data center accelerators, edge sensors.	AI/ML inference & training, DSP, streaming data pipelines, scientific computing.	Prototyping, low-volume products, real-time control, edge inference, research.

Forms of parallel execution in modern processors

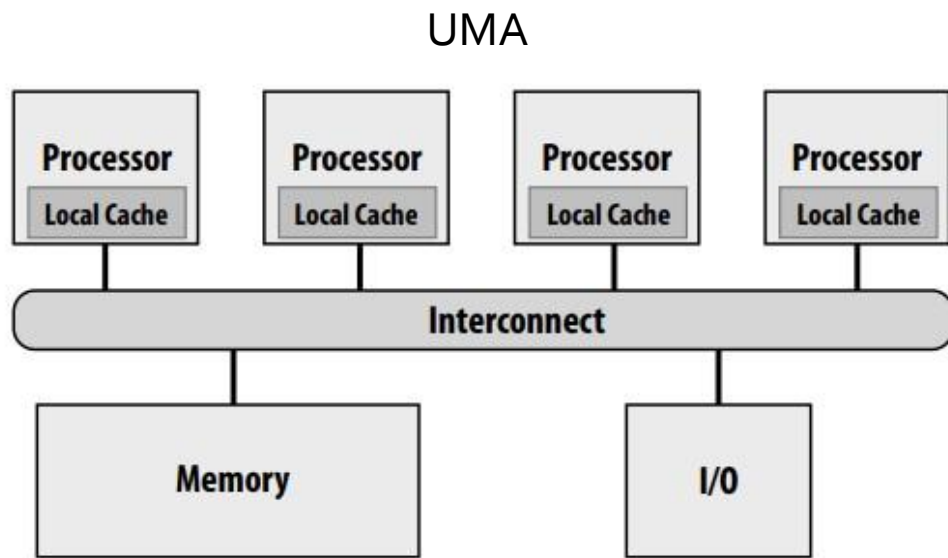
- Multi-cores
 - Multiple execution cores
 - Thread level parallelism
- SIMD
 - Multiple ALUs that execute the same instruction stream
 - Efficient design for data-parallel workloads
 - Dependencies must be known and specified beforehand
- Superscalar
 - Exploit ILP within an instruction stream
 - Parallelism automatically and dynamically discovered by the hardware during execution (not programmer visible)
- Hardware-supported multi-threading
 - Context of threads managed in HW
 - Execute instructions from several threads at the same time
- Specialized processing units that support parallelism

Heterogeneous Parallelism

- Scheduling problem: map processes on matching processing components
- Programming: design algorithms that decompose into components that match well with available resources
- HW design: establish a good mixture of resources for better performance

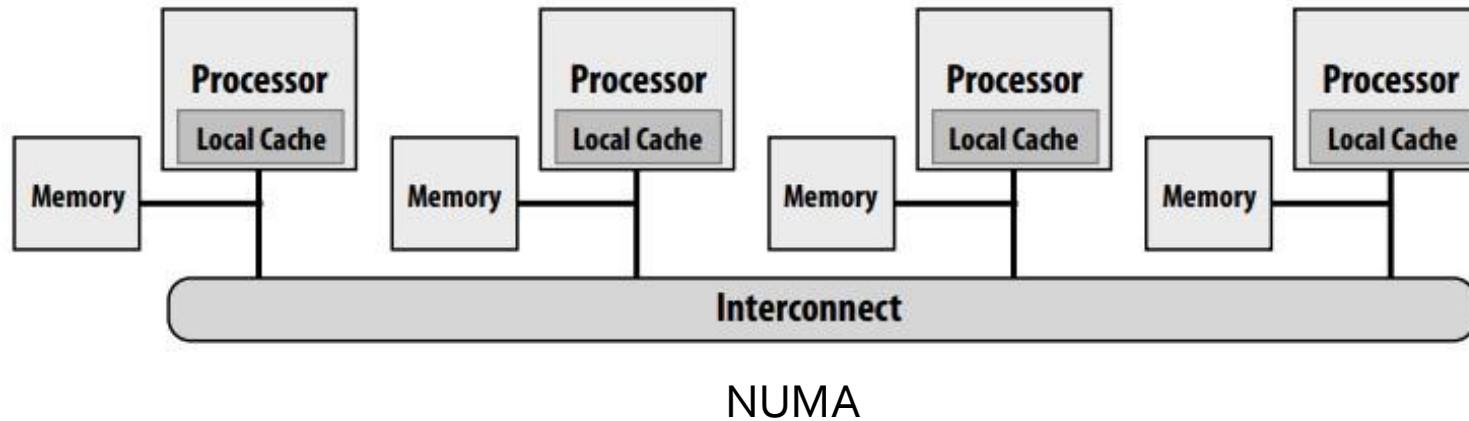
Parallel computer memory design

Uniform memory access model



- Shared memory computer
- Cost of accessing main memory is the same for all memory addresses (if no contention)
- Symmetric Multiprocessors

Non-uniform memory access model



- Physically distributed memory
- Memory access times are non-uniform (depends on whether the access is local or not)
- Shared memory over NUMA -> ccNUMA - cache coherence

Remote Direct Memory Access (RDMA)

■ What is RDMA?

- Technology that allows direct memory access from the memory of one computer into that of another without involving the operating system.
- Aims to improve the speed and efficiency of data transfer between computers in a network.

■ Features

- Low Latency: Reduces the time it takes to transfer data between systems.
- High Throughput: Increases the data transfer rate, making it ideal for high-performance computing.
- CPU Offloading: Minimizes CPU involvement in data transfer, freeing up resources for other tasks.
- Zero-Copy: Data is transferred directly between memory locations, avoiding unnecessary copying.

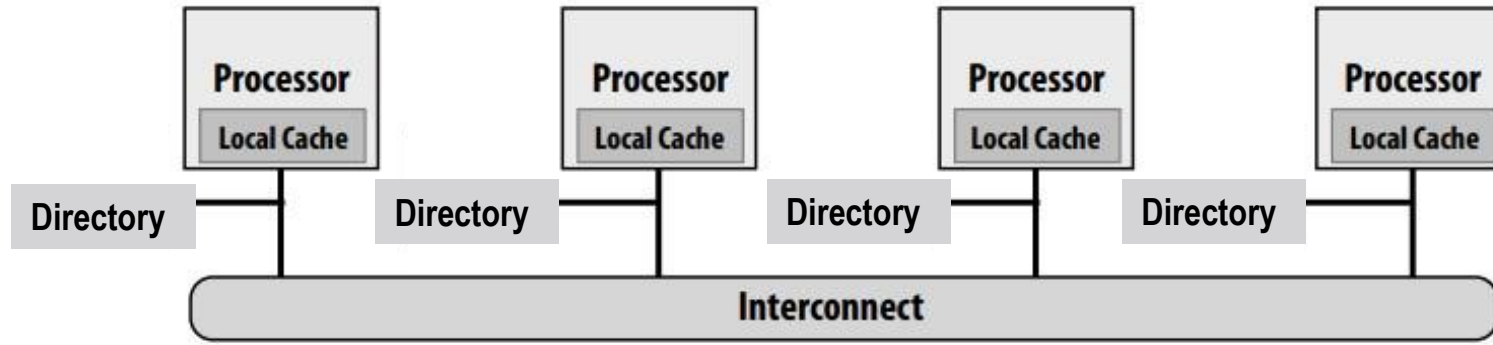
■ Applications

- High-Performance Computing (HPC): Used in supercomputers and data centres for efficient data transfer.
- Database Systems: Enhances the performance of distributed databases.
- Storage Networks: Improves the efficiency of storage area networks (SANs) and network-attached storage (NAS).

■ Benefits

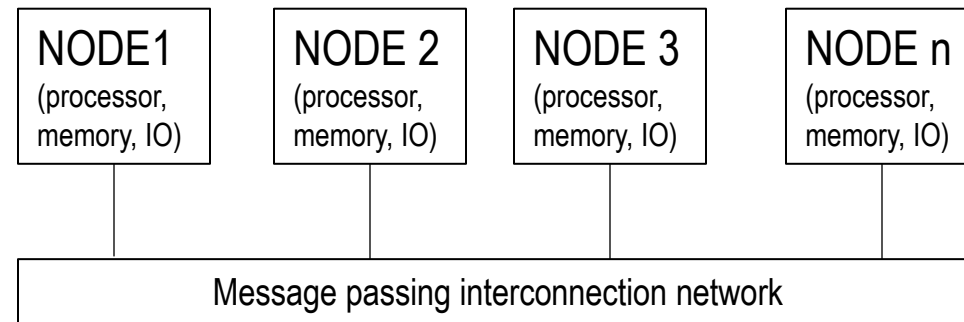
- Efficiency: Reduces CPU load and improves overall system performance.
- Scalability: Supports large-scale data transfer operations.
- Reliability: Provides consistent and reliable data transfer.

Cache only memory architecture (COMA)



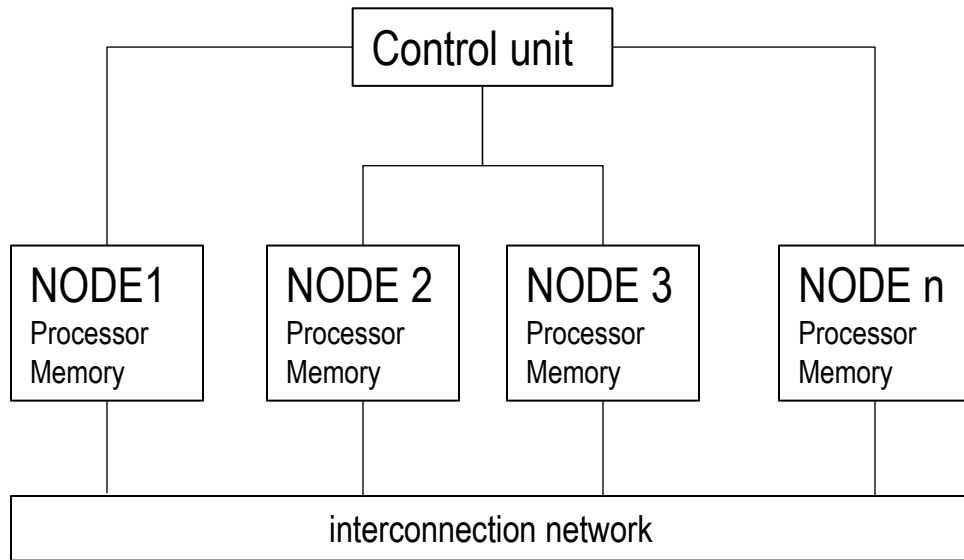
- A type of ccNUMA
- Each shared-memory module in the machine is a cache
- Automatically replicates the data and migrates it to the memory module of the node that is currently accessing it

No-Remote Memory Access (NORMA)



- Address space globally is not unique and the memory is not globally accessible by the processors
- Access memory on other nodes - only message passing

SIMD computers



- Exploit data parallelism
- Each unit performs the exact same instruction at a given moment
- SIMD computers -> SIMD processing units

Additional bibliography

- <http://www.lighterra.com/papers/modernmicroprocessors/>